PROJET KARATE

Le projet karaté va permettre à la ligue De Lorraine de Karaté d'organiser tout au long de l'année des compétitions de «Kata» dans les différents clubs qui lui sont affiliés et de numériser l'ensemble des informations pour une gestion plus simple et rapide

Contexte

Ce projet, réalisé dans le cadre de mon BTS SIO au Lycée Suzanne Valadon, vise à développer une application pour faciliter la gestion des clubs et la consultation des adhérents.

Ma mission avait les objectif suivant :
T1.2 Gestion des clubs : Cette tâche doit permettre la création, la modification et la suppression d'un club à partir d'un formulaire. Il faudra réaliser la maquette de ce formulaire, le faire valider par le chef de projet avant de commencer le codage.

T1.5 Consultation pour chaque club de la liste de ses adhérents triés par année de naissance et par ordre alphabétique : Pour chaque club, on souhaite l'affichage d'un tableau dans lequel figureront pour chaque membre le numéro de licence, le nom, le prénom, la date de naissance, l'adresse. Là aussi, il est conseillé de proposer une maquette au chef de projet avant de commencer le codage.

Menu général : Concevoir le menu général de l'application. Ce menu doit être conforme à la maquette conçue par l'équipe au complet au début de ce projet.

Outil utilisé

Plusieurs pistes pour la réalisation de cette mission sont possible :

- Une application web simple avec formulaire HTML/PHP
- Une application desktop en Java ou C#
- Une application desktop légère avec interface graphique simple (Visual Studio Code avec framework adapté)

La solution qui nous a été imposé dans le cadre de l'acquisition de nouvelle compétence sur une application desktop avec interface graphique sous Visual Studio Code, permettant un accès rapide et une manipulation aisée des données via une interface utilisateur intuitive.

L'interface a été développée avec Visual Studio Code 2022 avec un développement dans le langage C#

La base de données était quand à elle stocker sur phpmyadmin :

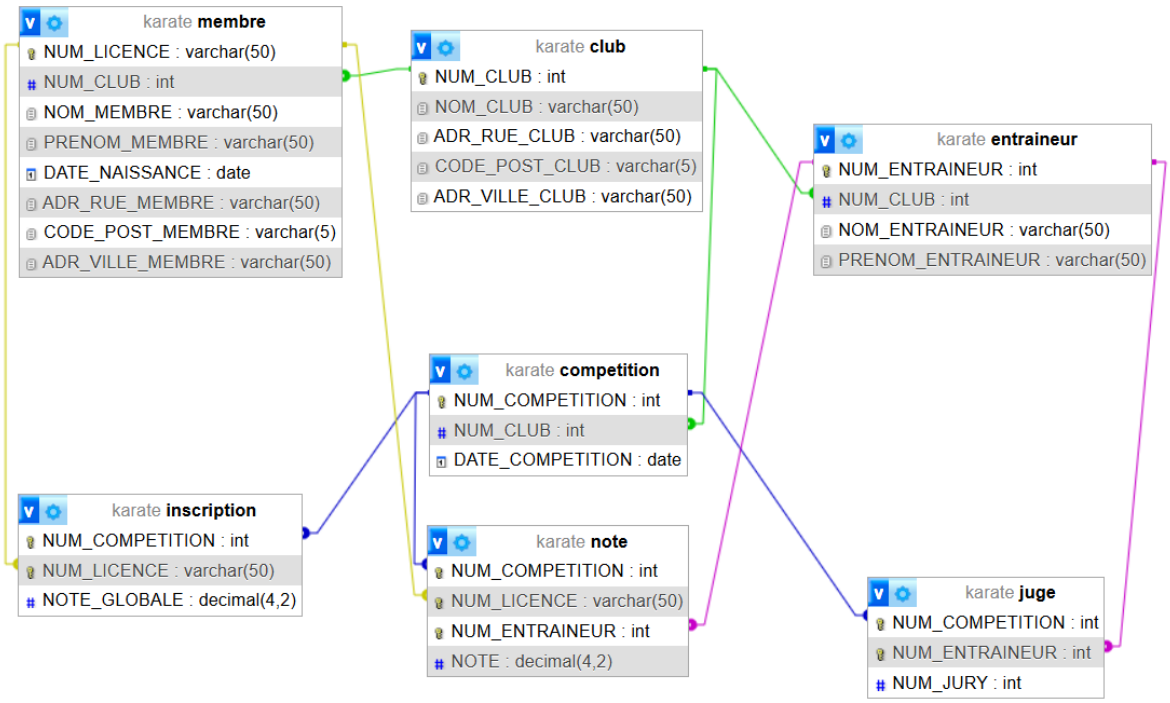
Module de connexion a la Base De Données (BDD)

Nous avons procéder à la création d'un fichier connexion.cs pour avoir accès à la BDD. Nous avons Modifier les valeurs de SERVER DATABASE UID et PASSWORD dans connectionString :

```
using MySql.Data.MySqlClient;
using System;
using System.Collections.Generic;
using System.Data.SqlClient;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace KarateM2
{
    10 références
    internal class Connexion
    {
        10 références
        public static MySqlConnection ConnectBD()
        {
            string connectionString = "SERVER=localhost; DATABASE=karate; UID=root; PASSWORD=";
            MySqlConnection conn = new MySqlConnection(connectionString);
            return conn;
        }
    }
}
```

Voici la structure de la base de données à laquelle nous nous somme connecté :

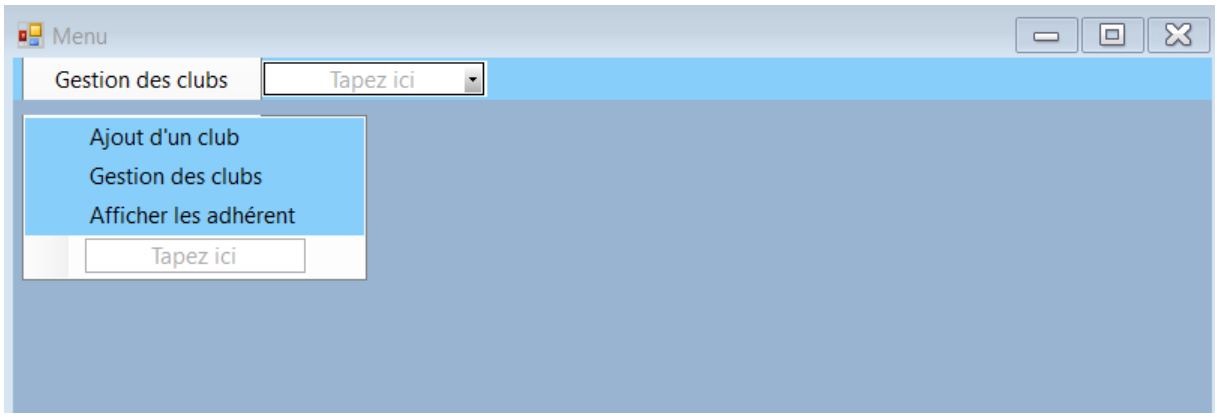


Réalisation des missions

Menu général

Rappelle de l'objectif

L'objectif est de créer un menu permettant de parcourir facilement les différentes fonctionnalité de l'application :



Pour la réalisation de l'interface graphique j'ai pue compter sur la boite à outil fourni par Visual Studio Code 2022 avec le bloc MenuStrip

Lorsque l'on clique sur l'une des fonctionnalité du menu une autre fenêtre s'ouvre pour afficher la page pour cela dans l'éditeur graphique en cliquant sur la fonctionnalité un évènement de clic est automatiquement créer dans l'éditeur de code on procède ensuite à la création d'une nouvelle instance du formulaire puis on affiche la fenêtre avec nom_de_l_instance.show et on répète cela pour les 3 fonctionnalité à développé dans ma mission :

```
private void ajoutDunClubToolStripMenuItem_Click(object sender, EventArgs e)
{
    AjoutClub Ajout = new AjoutClub();
    Ajout.Show();
}

1 référence
private void modifierUnClubToolStripMenuItem_Click(object sender, EventArgs e)
{
    GestionClub Gestion = new GestionClub();
    Gestion.Show();
}

1 référence
private void afficherLesAdhérentToolStripMenuItem_Click(object sender, EventArgs e)
{
    Consultation_adhérents consultation = new Consultation_adhérents();
    consultation.Show();
}
```

Gestion des clubs (T1.2)

Rappelle de l'objectif :

Cette tâche doit permettre la création, la modification et la suppression d'un club à partir d'un formulaire.

Cette fonctionnalité sera séparer en 2 formulaire un pour l'ajout de nouveau club et un autre pour la modification et la suppression de club déjà existant.

Ajout de nouveau club

J'ai commencer par la création de l'interface graphique une fois de plus à l'aide de la boite à outil et des outil suivant :

TextBox

Label

Button

Voici l'interface graphique réalisé :

Dans les propriété des différents bloc il est possible de modifié leur nom pour mieux les différencié dans l'éditeur de code au travers du champ name par exemple pour le champ identifiant :

(Name)	**textID**

J'ai ensuite coder le code suivant :

```
private void buttonAjout_Click(object sender, EventArgs e)
{
    MySqlConnection conn = Connexion.ConnectBD();
    conn.Open();
    string requete = "INSERT INTO club VALUES (@id, @nom, @adresse, @cp, @ville)";
    MySqlCommand cmd = new MySqlCommand(requete, conn);
    cmd.Parameters.AddWithValue("@id", textID.Text);
    cmd.Parameters.AddWithValue("@nom", textNomClub.Text);
    cmd.Parameters.AddWithValue("@adresse", textAdresse.Text);
    cmd.Parameters.AddWithValue("@cp", textCP.Text);
    cmd.Parameters.AddWithValue("@ville", textVille.Text);
    cmd.ExecuteNonQuery();
    MessageBox.Show("Le club à bien été inséré.");
    conn.Close();
}
```

Explication :

En double cliquant sur le bouton Ajouter je crée un événement de clic pour ce même bouton.

```
private void buttonAjout_Click(object sender, EventArgs e)
{
```

J'établit une connexion à la base de données en appelant Connexion.ConnectBD() pour obtenir une instance de MySqlConnection, puis l'ouvre avec conn.Open().

```
MySqlConnection conn = Connexion.ConnectBD();
conn.Open();
```

Je prépare une requête SQL d'insertion dans la table club avec des paramètres, puis crée une commande MySqlCommand associée à cette requête et à la connexion conn.

```
string requete = "INSERT INTO club VALUES (@id, @nom, @adresse, @cp, @ville)";
MySqlCommand cmd = new MySqlCommand(requete, conn);
```

J'associe les valeurs saisies dans les champs du formulaire aux paramètres de la requête SQL au travers de ces lignes.

```
cmd.Parameters.AddWithValue("@id", textID.Text);
cmd.Parameters.AddWithValue("@nom", textNomClub.Text);
cmd.Parameters.AddWithValue("@adresse", textAdresse.Text);
cmd.Parameters.AddWithValue("@cp", textCP.Text);
cmd.Parameters.AddWithValue("@ville", textVille.Text);
```

Cette ligne exécute la requête d'insertion dans la base de données sans retour de résultat.

```
cmd.ExecuteNonQuery();
```

Je fait s'afficher un message pour confirmer l'ajout du club, puis la connexion à la base de données est fermée.

Modification et suppression de club

J'ai utilisé les outil suivant pour la création de l'interface graphique.

DataGridView

ChargerDonnées

La méthode ChargerDonnées() récupère l'intégralité des données de la table club dans la base de données et les affiche dans un tableau (dataGridModif). Elle me servira à actualiser l'affichage des clubs, notamment au chargement du formulaire ou après une modification.

```
private void ChargerDonnees()
{
    dataGridModif.Rows.Clear();
    MySqlConnection conn = Connexion.ConnectBD();
    conn.Open();
    string requete = "SELECT * FROM club";
    MySqlCommand cmd = new MySqlCommand(requete, conn);
    MySqlDataReader reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        dataGridModif.Rows.Add(reader["NUM_CLUB"], reader["NOM_CLUB"], reader["ADR_RUE_CLUB"], reader["CODE_POST_CLUB"], reader["ADR_VILLE_CLUB"]);
    }
    conn.Close();
}
```

```
    dataGridModif.Rows.Clear();
```

Cette ligne vide le tableau d'affichage pour éviter la duplication des données à chaque actualisation.

```
MySqlConnection conn = Connexion.ConnectBD();
conn.Open();
string requete = "SELECT * FROM club";
MySqlCommand cmd = new MySqlCommand(requete, conn);
```

On établit une connexion à la base de données puis on prépare une requête SQL pour récupérer toutes les informations de la table club.

```
    MySqlDataReader reader = cmd.ExecuteReader();
```

Cette instruction exécute la requête SQL préparée et renvoie un objet MySqlDataReader. Cet objet permet de lire en avant, ligne par ligne, les résultats retournés par la base de données. Contrairement à une exécution qui renverrait toutes les données en mémoire d'un coup, MySqlDataReader offre une lecture efficace et en flux continu, ce qui est adapté pour parcourir des grands jeux de données sans surcharger la mémoire.

```
    while (reader.Read())
    {
        dataGridModif.Rows.Add(reader["NUM_CLUB"], reader["NOM_CLUB"], reader["ADR_RUE_CLUB"], reader["CODE_POST_CLUB"], reader["ADR_VILLE_CLUB"]);
    }
    conn.Close();
}
```

On utilise ce reader pour parcourir chaque enregistrement renvoyé et on ajoute ses informations dans le tableau d'affichage pour les rendre visibles à l'utilisateur. Et on ferme finalement la connexion.

```
private void GestionClub_Load(object sender, EventArgs e)
{
    ChargerDonnees();
}
```

Cette méthode est déclenchée automatiquement lors du chargement de la fenêtre de gestion des clubs. Son rôle est d'initialiser l'affichage en appelant la fonction ChargerDonnees(), ce qui permet de remplir immédiatement le tableau avec les données actuelles des clubs dès l'ouverture de l'interface. Cela garantit que l'utilisateur dispose directement des informations à jour sans action supplémentaire.

ViderChamps()

```
private void ViderChamps()
{
    textNom.Clear();
    textAdresse.Clear();
    textCP.Clear();
    textVille.Clear();
}
```

Cette méthode sert à réinitialiser les champs de saisie du formulaire en effaçant le contenu des zones de texte. Elle sera utilisée pour nettoyer l'interface après une opération (modification ou suppression) afin d'éviter que d'anciennes données restent affichées, ce qui pourrait induire en erreur l'utilisateur ou causer des erreurs lors d'actions ultérieures. Cela contribue à une meilleure expérience utilisateur en gardant l'interface claire et prête pour une nouvelle saisie.

dataGridModif_CellContentClick()

```
private void dataGridModif_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    DataGridViewRow row = dataGridModif.Rows[e.RowIndex];

    idSelectionne = Convert.ToInt32(row.Cells["Id"].Value);
    textNom.Text = row.Cells["Nom"].Value.ToString();
    textAdresse.Text = row.Cells["Adresse"].Value.ToString();
    textCP.Text = row.Cells["CP"].Value.ToString();
    textVille.Text = row.Cells["Ville"].Value.ToString();
}
```

Cette méthode gère l'événement de clic sur une cellule du dataGridModif. Son rôle est de récupérer les données de la ligne sélectionnée dans le tableau et de les afficher dans les champs de saisie du formulaire. Cela permet à l'utilisateur de visualiser et de modifier facilement les informations du club sélectionné.

```
private void dataGridModif_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    DataGridViewRow row = dataGridModif.Rows[e.RowIndex];
```

Ici, on récupère la ligne correspondant à la cellule cliquée grâce à l'index de la ligne (e.RowIndex).

```
    idSelectionne = Convert.ToInt32(row.Cells["Id"].Value);
    textNom.Text = row.Cells["Nom"].Value.ToString();
    textAdresse.Text = row.Cells["Adresse"].Value.ToString();
    textCP.Text = row.Cells["CP"].Value.ToString();
    textVille.Text = row.Cells["Ville"].Value.ToString();
}
```

On extrait ensuite les valeurs des cellules de cette ligne pour les affecter aux variables et aux champs de saisie correspondants, facilitant ainsi la modification ou la suppression du club sélectionné.

buttonModifier_Click()

Cette méthode est déclenchée lors du clic sur le bouton "Modifier". Elle permet de mettre à jour les informations d'un club sélectionné dans la base de données à partir des champs modifiés dans le formulaire. Après la mise à jour, les champs sont vidés et le tableau est rechargé avec les données actualisées.

```
private void buttonModifier_Click(object sender, EventArgs e)
{
    MySqlConnection conn = Connexion.ConnectBD();
    conn.Open();
    string requete = "UPDATE club SET NOM_CLUB = @Nom, ADR_RUE_CLUB = @Adresse, CODE_POST_CLUB = @CP, ADR_VILLE_CLUB = @Ville WHERE NUM_CLUB = @Numero";
    MySqlCommand cmd = new MySqlCommand(requete, conn);
    cmd.Parameters.AddWithValue("@Numero", idSelectionne);
    cmd.Parameters.AddWithValue("@Nom", textNom.Text);
    cmd.Parameters.AddWithValue("@Adresse", textAdresse.Text);
    cmd.Parameters.AddWithValue("@CP", textCP.Text);
    cmd.Parameters.AddWithValue("@Ville", textVille.Text);
    cmd.ExecuteNonQuery();
    conn.Close();
    MessageBox.Show("Club Modifié");
    ViderChamps();
    ChargerDonnees();
}
```


```
private void buttonModifier_Click(object sender, EventArgs e)
{
    MySqlConnection conn = Connexion.ConnectBD();
    conn.Open();
    string requete = "UPDATE club SET NOM_CLUB = @Nom, ADR_RUE_CLUB = @Adresse, CODE_POST_CLUB = @CP, ADR_VILLE_CLUB = @Ville WHERE NUM_CLUB = @Numero";
    MySqlCommand cmd = new MySqlCommand(requete, conn);
```


On établit une connexion à la base de données et on prépare une requête UPDATE pour modifier les données du club identifié par son numéro (NUM_CLUB).

```
    cmd.Parameters.AddWithValue("@Numero", idSelectionne);
    cmd.Parameters.AddWithValue("@Nom", textNom.Text);
    cmd.Parameters.AddWithValue("@Adresse", textAdresse.Text);
    cmd.Parameters.AddWithValue("@CP", textCP.Text);
    cmd.Parameters.AddWithValue("@Ville", textVille.Text);
```

On lie les valeurs saisies dans les champs du formulaire aux paramètres de la requête SQL, ce qui permet de sécuriser l'injection de données.

```
    cmd.ExecuteNonQuery();
    conn.Close();
    MessageBox.Show("Club Modifié");
```

La requête est exécutée sans retour de résultat (ExecuteNonQuery), on ferme la connexion puis on informe l'utilisateur de la réussite de la modification.

```
    ViderChamps();
    ChargerDonnees();
}
```

On utilise les méthodes créées précédemment pour vider les champs du formulaire, puis on recharge les données dans le tableau afin d'afficher les modifications à jour.

buttonDelete_Click()

Ce bloc est exécuté lorsqu'on clique sur le bouton de suppression. Il permet de supprimer un club de la base de données en se basant sur les informations saisies dans le formulaire (nom, adresse, code postal et ville). Une fois la suppression effectuée, la liste des clubs est rechargée pour refléter les changements.

```
private void buttonDelete_Click(object sender, EventArgs e)
{
    MySqlConnection conn = Connexion.ConnectBD();
    conn.Open();
    string requete = "DELETE FROM club WHERE NOM_CLUB = @Nom AND ADR_RUE_CLUB = @Adresse AND CODE_POST_CLUB = @CP AND ADR_VILLE_CLUB = @Ville";
    MySqlCommand cmd = new MySqlCommand(requete, conn);
    cmd.Parameters.AddWithValue("@Nom", textNom.Text);
    cmd.Parameters.AddWithValue("@Adresse", textAdresse.Text);
    cmd.Parameters.AddWithValue("@CP", textCP.Text);
    cmd.Parameters.AddWithValue("@Ville", textVille.Text);
    cmd.ExecuteNonQuery();
    conn.Close();
    MessageBox.Show("Club Supprimé");
    ChargerDonnees();
}
```

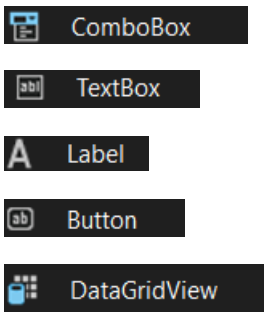
Elle a fonctionnement similaire à la précédente la différence se trouve dans le fait que c'est une requête DELETE à la place d'une UPDATE.

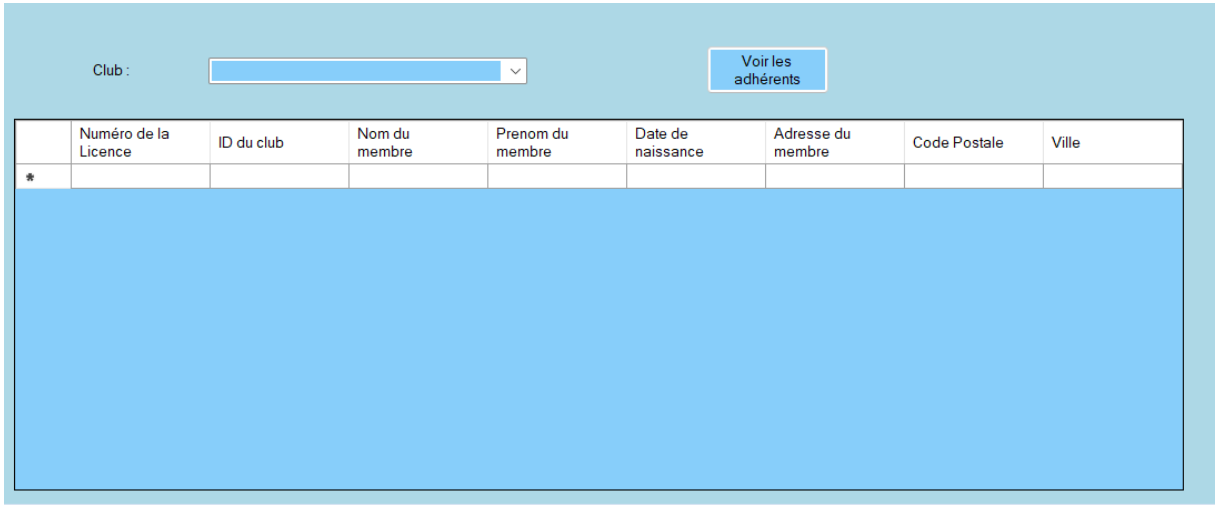
Consultation des adhérents (T1.5)

Rappelles de l'objectif

Pour chaque club, on souhaite l'affichage d'un tableau dans lequel figureront pour chaque membre le numéro de licence, le nom, le prénom, la date de naissance, l'adresse. Il faudra les trié par date de naissance et par ordre alphabétique

J'ai utilisé les outils suivant :





Consultation_adhérents_Load()

Cette méthode est exécutée automatiquement lors du chargement de la fenêtre de consultation des adhérents. Elle permet de récupérer tous les clubs présents en base de données et d'ajouter leurs noms dans la ListBox ListeClub, afin de permettre à l'utilisateur de sélectionner un club pour ensuite consulter ses adhérents.

```
private void Consultation_adhérents_Load(object sender, EventArgs e)
{
    ListeClub.Items.Clear();
    MySqlConnection conn = Connexion.ConnectBD();
    conn.Open();
    string requete = "SELECT NUM_CLUB, NOM_CLUB FROM club";
    MySqlCommand cmd = new MySqlCommand(requete, conn);
    MySqlDataReader reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        ListeClub.Items.Add(reader["NOM_CLUB"].ToString());
    }
    conn.Close();
}
```

```
    ListeClub.Items.Clear();
```

Avant de remplir la liste, on vide son contenu pour éviter les doublons si l'utilisateur rouvre le formulaire plusieurs fois.

```
    MySqlConnection conn = Connexion.ConnectBD();
    conn.Open();
    string requete = "SELECT NUM_CLUB, NOM_CLUB FROM club";
    MySqlCommand cmd = new MySqlCommand(requete, conn);
```

Connexion à la base de données et préparation d'une requête SQL qui sélectionne le numéro et le nom de tous les clubs.

```
    MySqlDataReader reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        ListeClub.Items.Add(reader["NOM_CLUB"].ToString());
    }
    conn.Close();
}
```

On exécute la requête, puis on parcourt les résultats pour ajouter chaque nom de club à la liste déroulante. La connexion est ensuite fermée proprement.

Bouton1_Click()

```
private void button1_Click(object sender, EventArgs e)
{
    dataGridVoirAd.Rows.Clear();
    MySqlConnection conn = Connexion.ConnectBD();
    conn.Open();
    string requete = "SELECT * FROM membre m INNER JOIN club c ON m.NUM_CLUB = c.NUM_CLUB" +
        " WHERE NOM_CLUB = @choixclub ORDER BY m.DATE_NAISSANCE, m.NOM_MEMBRE";
    MySqlCommand cmd = new MySqlCommand(requete, conn);
    cmd.Parameters.AddWithValue("@choixclub", ListeClub.SelectedItem.ToString());
    MySqlDataReader reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        dataGridVoirAd.Rows.Add(reader["NUM_LICENCE"], reader["NUM_CLUB"], reader["NOM_MEMBRE"], reader["PRENOM_MEMBRE"],
            reader["DATE_NAISSANCE"], reader["ADR_RUE_MEMBRE"], reader["CODE_POST_MEMBRE"], reader["ADR_VILLE_MEMBRE"]);
    }
    conn.Close();
}
```

Cette méthode est déclenchée lors du clic sur le bouton permettant d'afficher les adhérents. Elle vide le tableau d'affichage puis exécute une requête SQL qui récupère tous les membres liés au club sélectionné dans la liste déroulante, les trie par date de naissance puis par nom, et les insère dans la DataGridView.

```
private void button1_Click(object sender, EventArgs e)
{
    dataGridVoirAd.Rows.Clear();
    MySqlConnection conn = Connexion.ConnectBD();
    conn.Open();
```

Cette portion initialise l'exécution lors du clic sur le bouton. Elle commence par vider le tableau d'affichage pour éviter les doublons, puis établit une connexion à la base de données.

```
    string requete = "SELECT * FROM membre m INNER JOIN club c ON m.NUM_CLUB = c.NUM_CLUB" +
        " WHERE NOM_CLUB = @choixclub ORDER BY m.DATE_NAISSANCE, m.NOM_MEMBRE";
    MySqlCommand cmd = new MySqlCommand(requete, conn);
    cmd.Parameters.AddWithValue("@choixclub", ListeClub.SelectedItem.ToString());
```

Préparation de la requête SQL : elle sélectionne tous les membres d'un club choisi par l'utilisateur en associant les tables membre et club. La clause ORDER BY permet d'organiser les résultats d'abord par date de naissance puis par nom. Le paramètre @choixclub est injecté à partir de l'élément sélectionné dans la liste.

```
    MySqlDataReader reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        dataGridVoirAd.Rows.Add(reader["NUM_LICENCE"], reader["NUM_CLUB"], reader["NOM_MEMBRE"], reader["PRENOM_MEMBRE"],
            reader["DATE_NAISSANCE"], reader["ADR_RUE_MEMBRE"], reader["CODE_POST_MEMBRE"], reader["ADR_VILLE_MEMBRE"]);
    }
    conn.Close();
}
```

Exécution de la requête et parcours des résultats pour les afficher ligne par ligne dans la DataGridView. Une fois tous les adhérents affichés, la connexion à la base est fermée.

Conclusion

Ce projet m'a permis de développer une application fonctionnelle répondant aux besoins de la ligue De Lorraine de Karaté pour la gestion des clubs et de leurs adhérents.
J'ai pu appréhender des compétences comme la conception d'interface via des blocs, manipulation de base de données et programmation structurée.
Ce travail m'a aussi confronté à des défis techniques que j'ai appris à résoudre, notamment la gestion dynamique des interfaces.